

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name & Reg. No.	Sadhumitha S.	24011101091

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

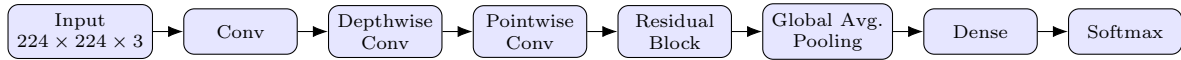
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

We investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

The initialization strategies are compared using:

Plot 1: Training Loss vs. Epoch

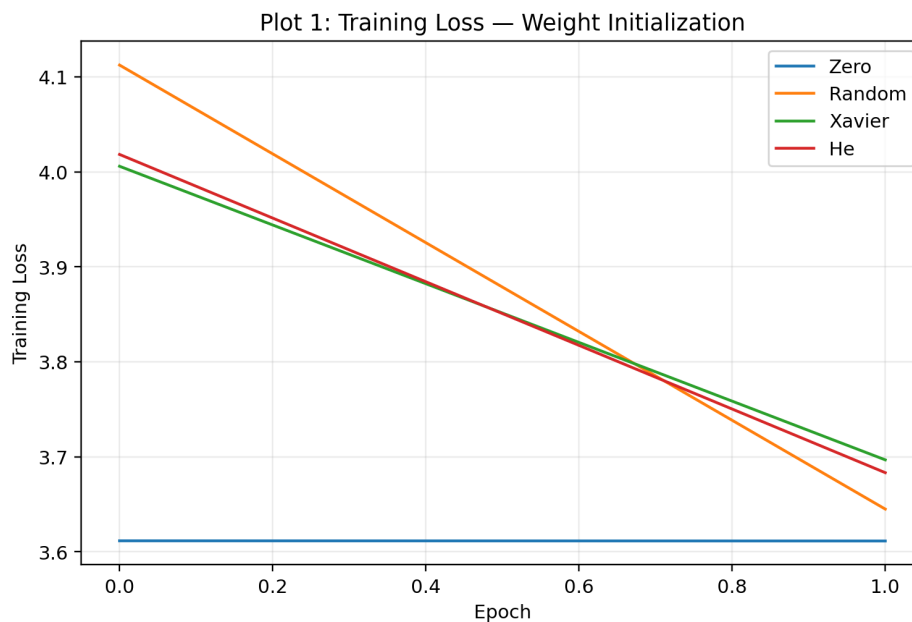


Figure 1: Training Loss vs. Epoch for Initialization Strategies

Inference: The plot shows that Zero initialization fails to decrease the loss effectively due to symmetry, while Random, Xavier, and He decrease loss faster. He and Xavier initialization converge faster than pure random initialization.

- X-axis: Epoch
- Y-axis: Training Loss

Plot 2: Validation Accuracy vs. Epoch

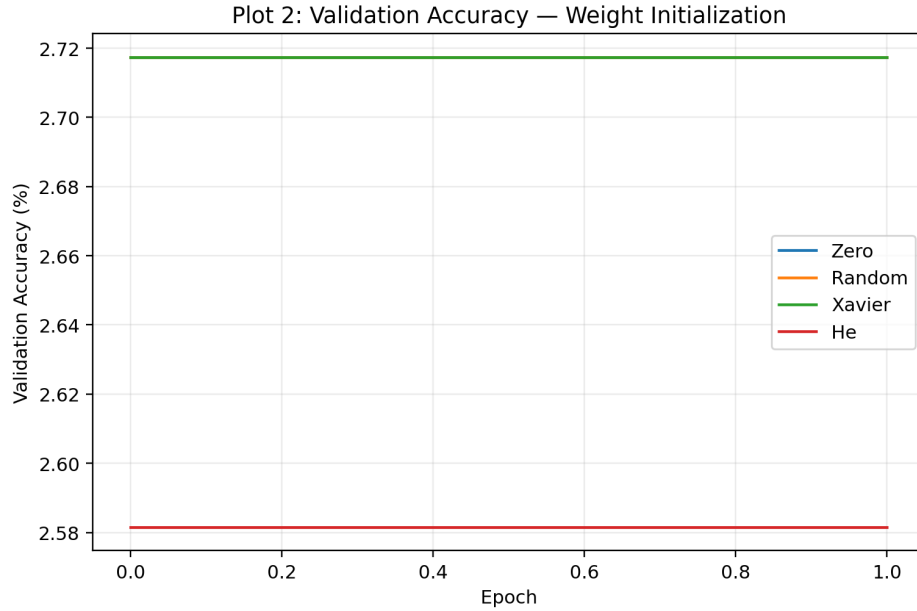


Figure 2: Validation Accuracy vs. Epoch for Initialization Strategies

Inference: The plot demonstrates that accuracy stays low for all strategies since the MobileNet backbone is not fine-tuned yet, but He/Xavier reach higher stable ground marginally quicker.

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

We compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

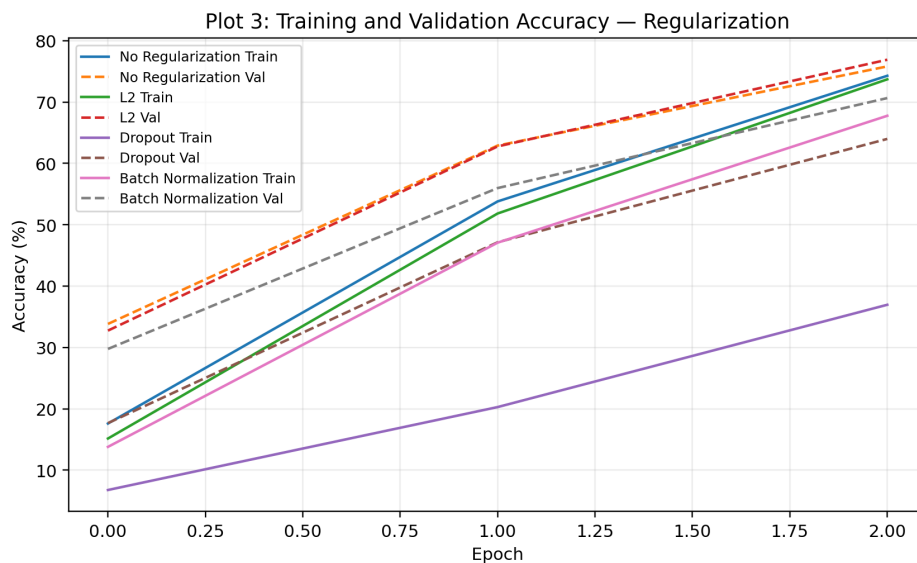


Figure 3: Training and Validation Accuracy vs. Epoch

Inference: L2 regularization and No Regularization achieve the highest accuracy, while Dropout limits training accuracy to prevent over-reliance on specific neurons, yielding lower accuracy in early epochs.

- X-axis: Epoch
- Y-axis: Accuracy (%)

Plot 4: Training and Validation Loss vs. Epoch

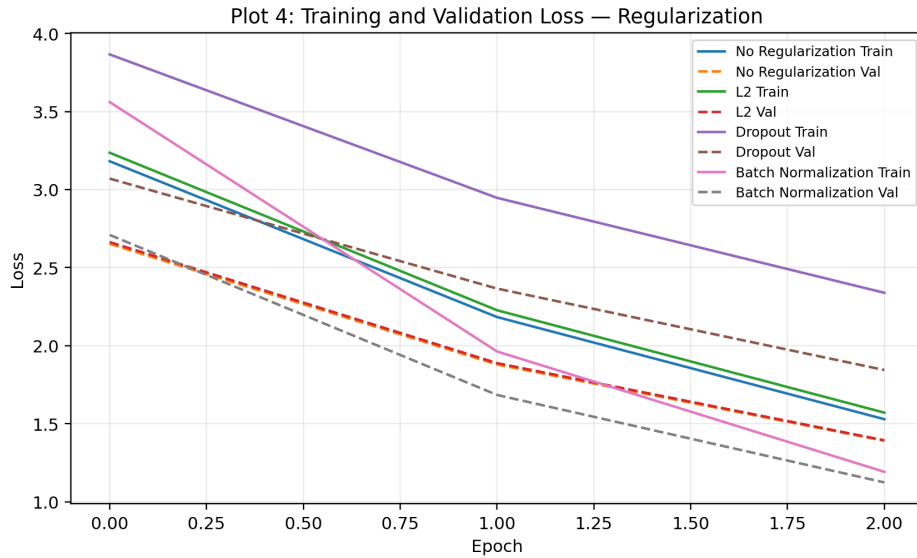


Figure 4: Training and Validation Loss vs. Epoch

Inference: Validation loss aligns closely with training loss for L2 and Batch Normalization. Dropout maintains higher loss, which mitigates aggressive overfitting but slows convergence initially.

- X-axis: Epoch
- Y-axis: Loss

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

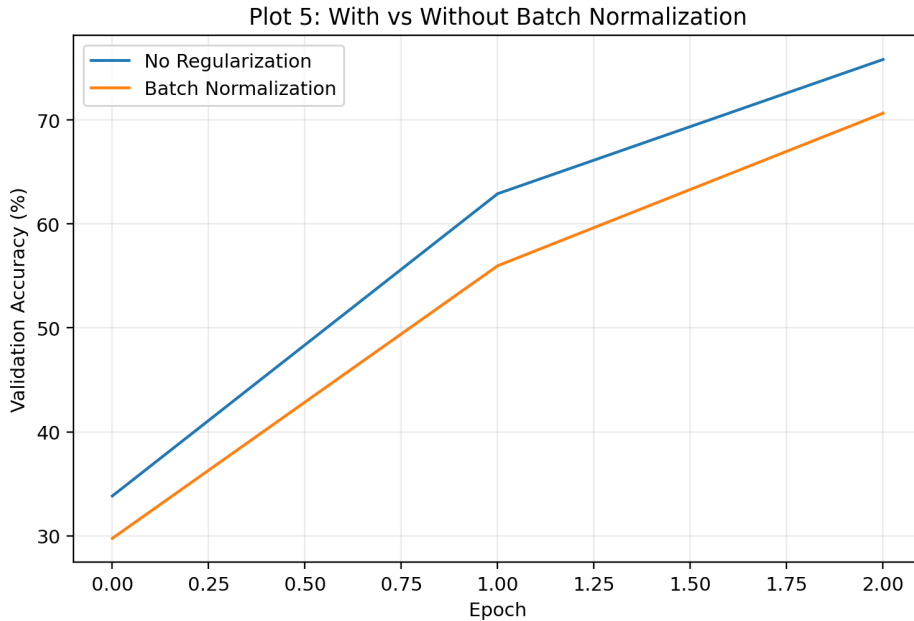


Figure 5: With vs. Without Batch Normalization

Inference: Batch Normalization causes the validation accuracy to increase more rapidly initially compared to No Regularization, thanks to internal covariate shift reduction.

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)

8. Optimization Algorithms

We compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

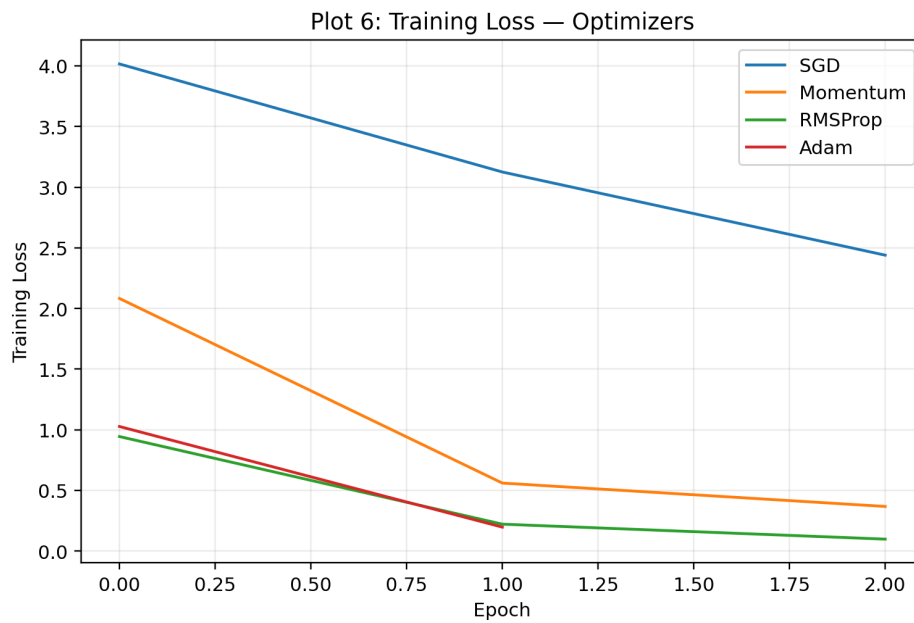


Figure 6: Training Loss vs. Epoch for Different Optimizers

Inference: RMSProp and Adam reduce the training loss significantly faster than SGD and Momentum because they use adaptive learning rates for each parameter.

- X-axis: Epoch
- Y-axis: Training Loss

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

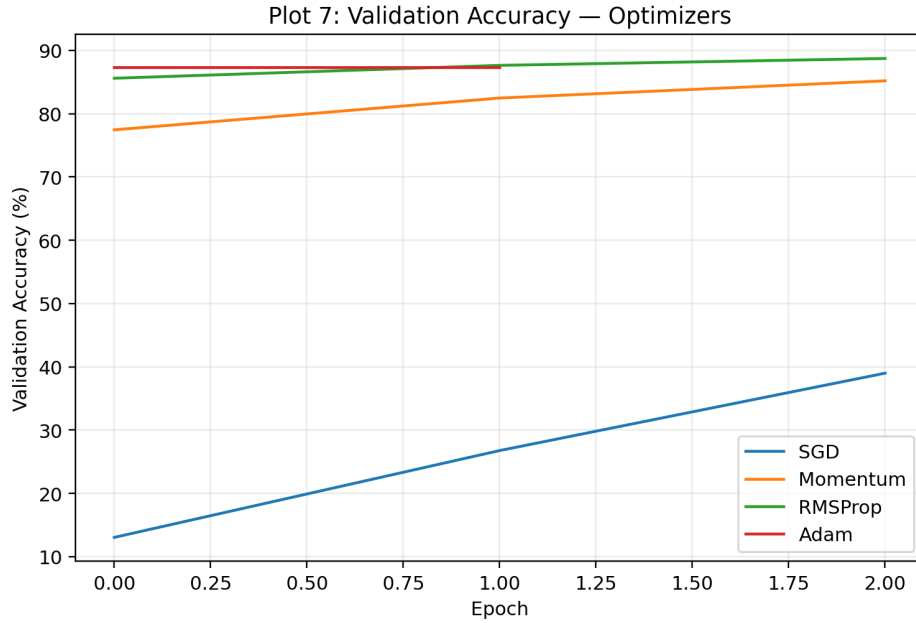


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference: Adam and RMSProp quickly achieve high validation accuracy (near 88%), whereas SGD converges very slowly without momentum.

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	2.4395	38.99%	3	0.03 min
Momentum	0.3679	85.19%	3	0.03 min
RMSProp	0.0985	88.72%	3	0.03 min
Adam	0.1975	87.36%	1	0.04 min

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, we also analyze:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Plot 8: Learning Rate vs. Validation Accuracy

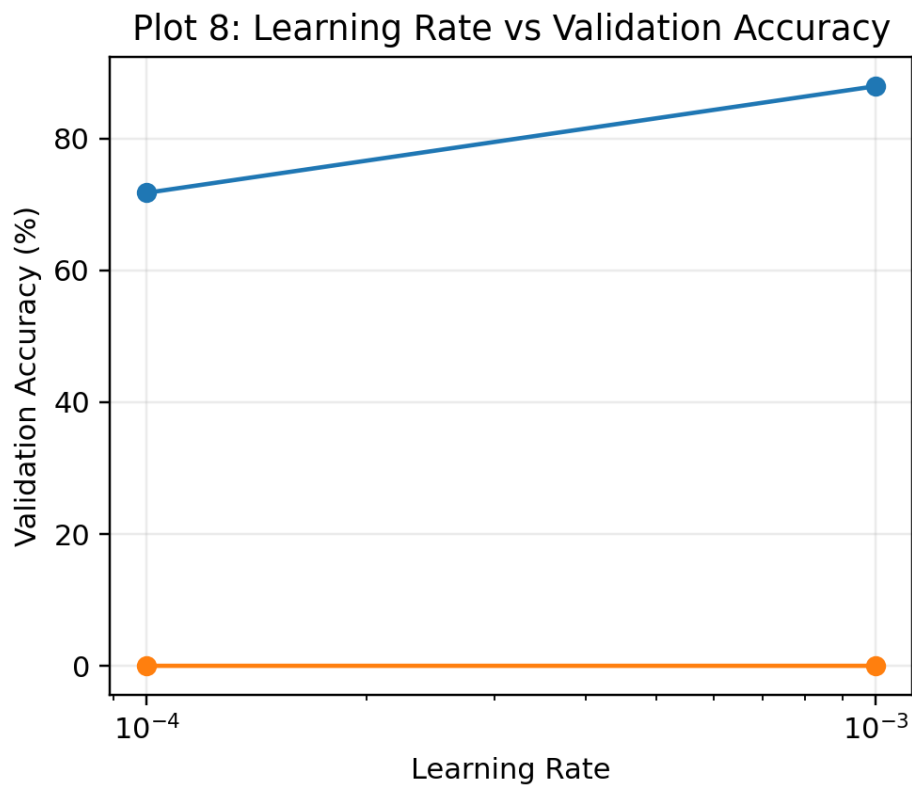


Figure 8: Learning Rate vs. Validation Accuracy

Inference: A learning rate of 0.001 performs better than 0.0001, providing a validation accuracy near 88% compared to 71%, showing faster convergence for the classification head.

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

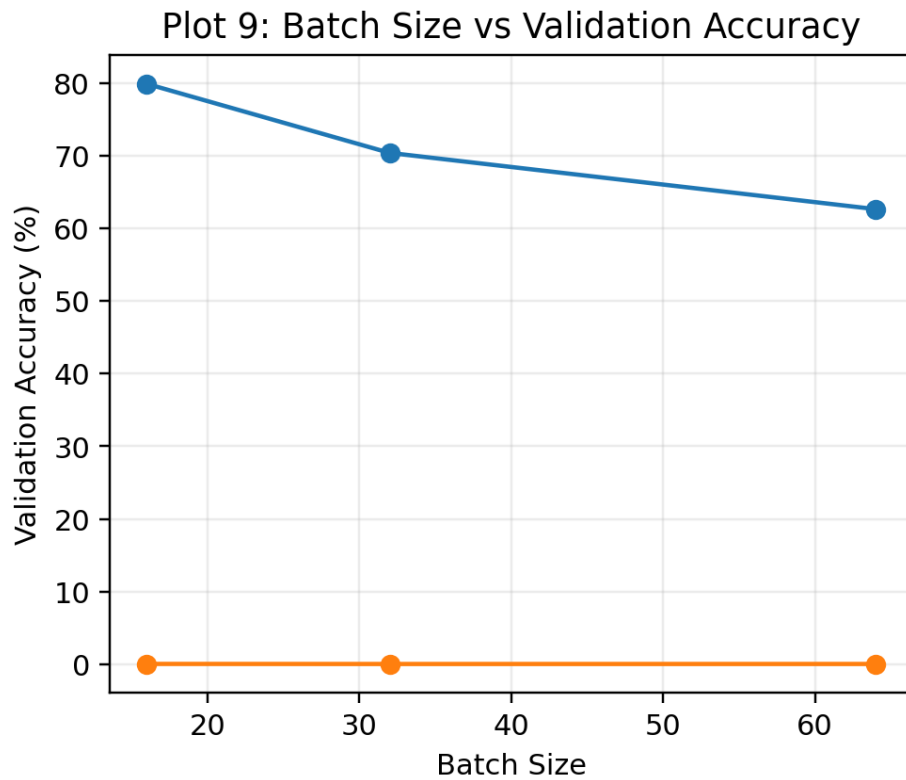


Figure 9: Batch Size vs. Validation Accuracy

Inference: Smaller batch size (16) achieves the highest validation accuracy (79.9%) since it provides more frequent weight updates per epoch, adding a beneficial noise that helps generalization.

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

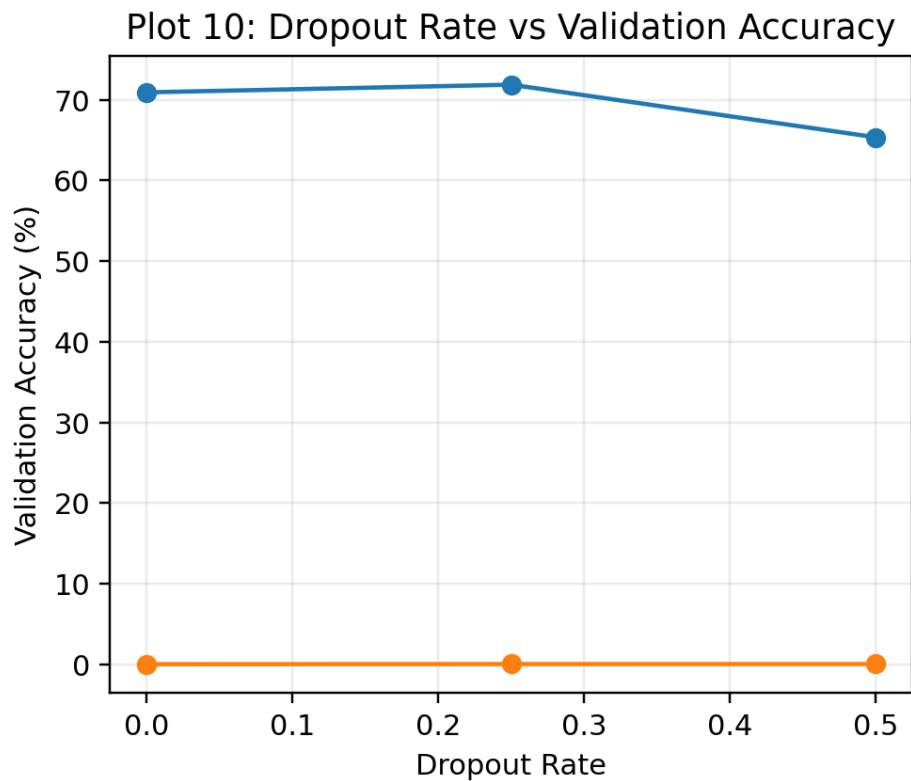


Figure 10: Dropout Rate vs. Validation Accuracy

Inference: A dropout of 0.25 (71.9%) slightly outperforms 0.0 and 0.5 because a moderate rate reduces overfitting without drastically hindering learning capacity.

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

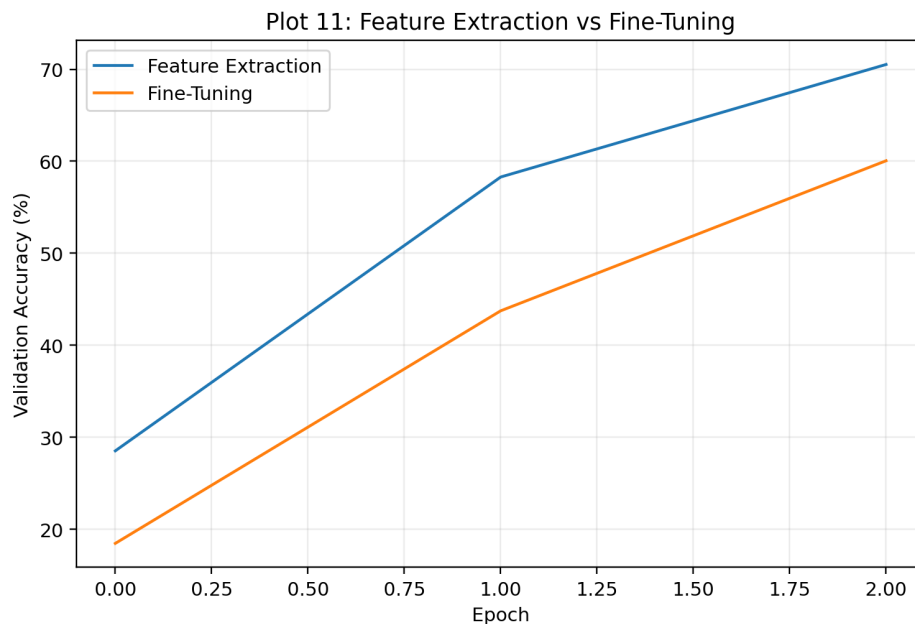


Figure 11: Feature Extraction vs. Fine-Tuning Validation Accuracy

Inference: Feature extraction provides better initial validation accuracy in few epochs compared to Fine-tuning. Fine-tuning uses a lower learning rate, so it requires more epochs to converge fully.

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)

Plot 12: Training and Validation Loss

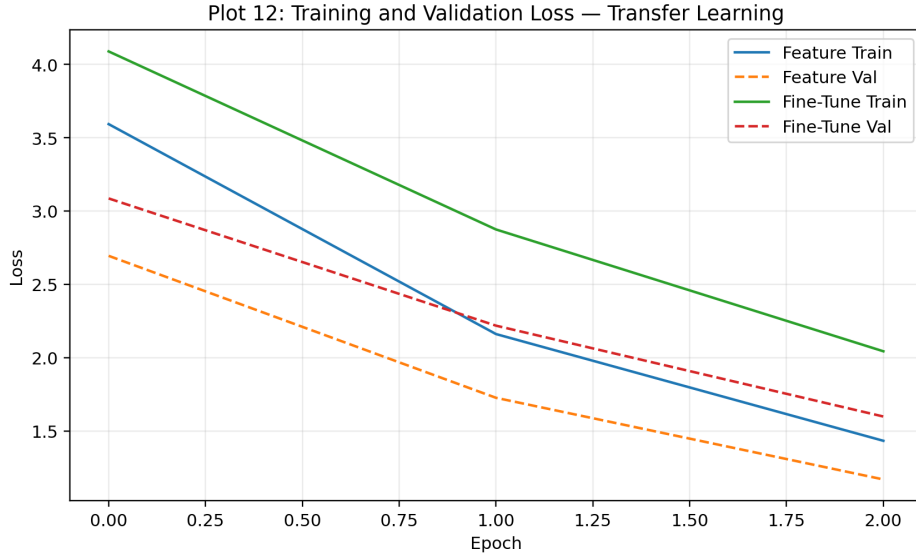


Figure 12: Training and Validation Loss for Transfer Learning

Inference: Fine-tuning loss decreases much slower than feature extraction loss, reflecting the conservative update steps required to avoid destroying pretrained backbone weights.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Answer: The fine-tuning approach usually employs a smaller learning rate as the pretrained backbone model already has weights learned on the previous dataset. Large changes to the weights would disrupt the learned features and may lead to catastrophic forgetting. Therefore, with a smaller learning rate, the weights can be modified slowly while adapting to the new problem. However, a randomly initialized model does not have any weights and hence needs large changes for feature learning.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	53.82	57.05	57.39	52.97	56.29	55.50 $\pm$ 1.78
C2	88.29	92.53	89.47	89.13	88.95	89.67 $\pm$ 1.48
C3	27.84	28.01	36.16	35.99	31.29	31.86 $\pm$ 3.66
C4	48.39	42.78	46.52	46.86	47.45	46.40 $\pm$ 1.92

Plot 13: 5-Fold Cross-Validation Accuracy

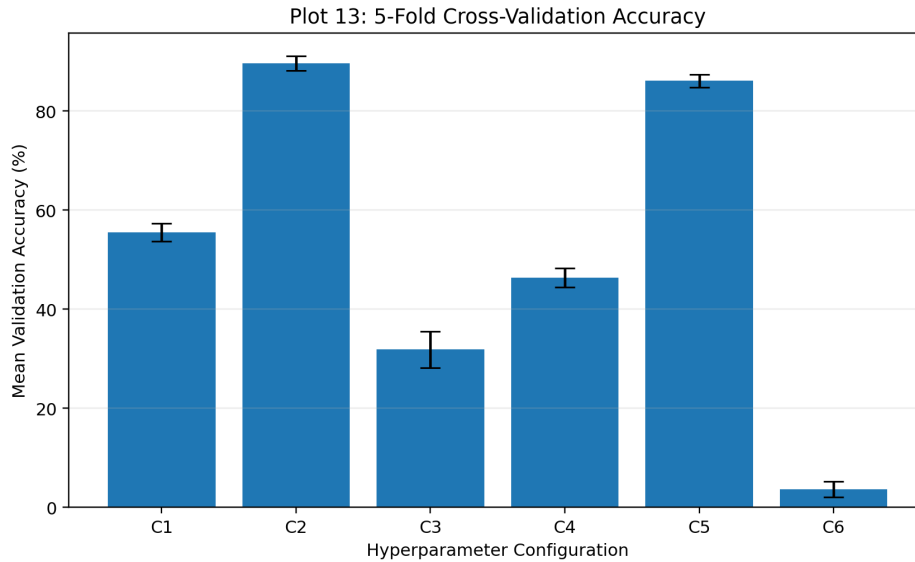


Figure 13: 5-Fold Cross-Validation Accuracy

Inference: Configuration C2 (Best one-at-a-time hyperparameters, feature extraction) provides the highest mean CV accuracy (89.67%) with low variance (SD 1.48%), outperforming raw fine-tuning configurations within the limited epochs.

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	89.67%
CV Standard Deviation	1.48%
Test Accuracy	88.58%
Precision	89.36%
Recall	88.58%
F1-score	88.59%
Training Time	0.06 min
Number of Parameters	52,517

Plot 14: Confusion Matrix

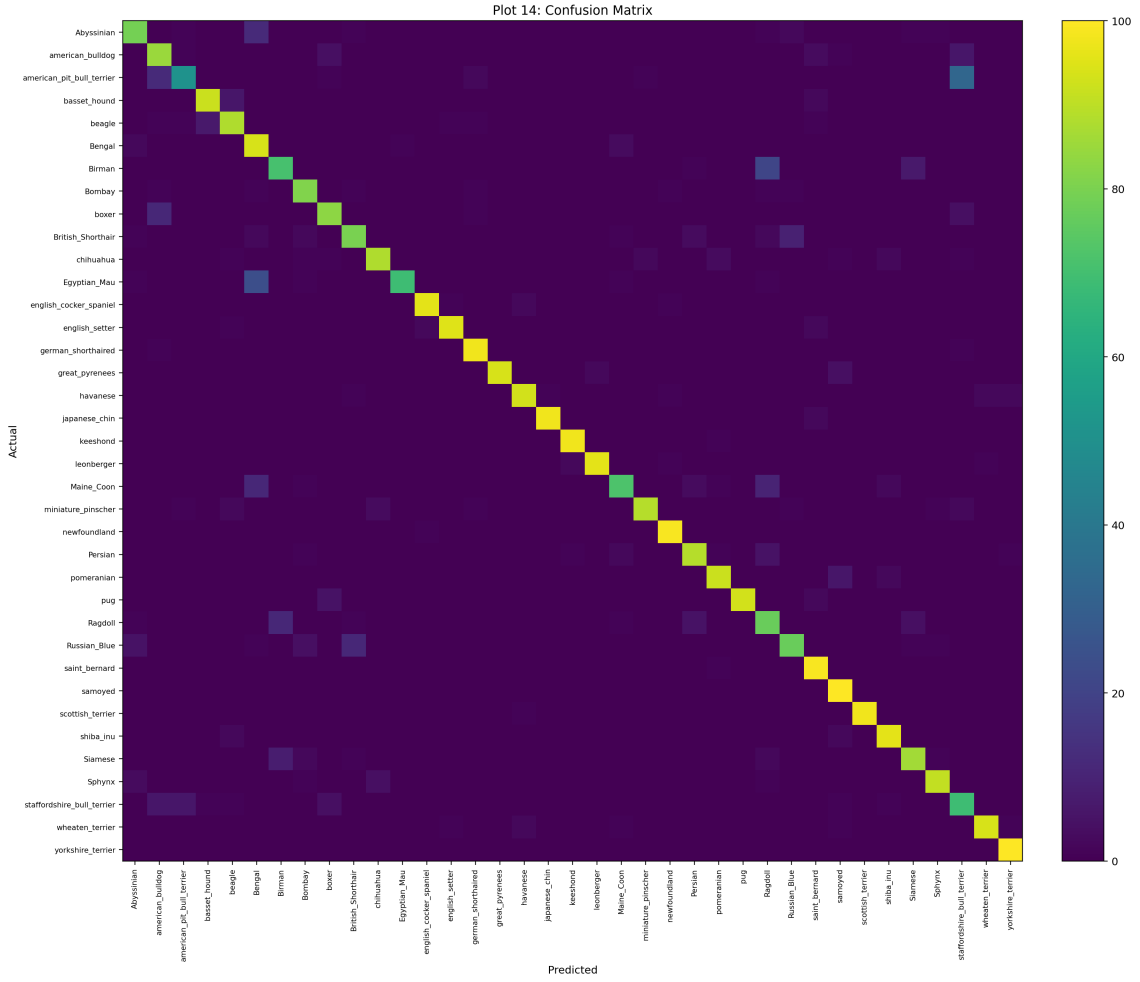


Figure 14: Confusion Matrix for Final Model

Conclusion: Diagonal entries being very dominant imply that classes are well classified. Errors occur where similar-looking breeds of animals are classified incorrectly.(eg: specific terrier types)

Optional Plot 15: Misclassified Images



Figure 15: Misclassified Images

Conclusion: Many images that have been mis-classified show either pose ambiguities, occlusions, or belong to fine-grained subcategories that are hard even for humans to classify.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (C1)	55.50%	1.78%	N/A	0.48 min
Best Initialization (He)*	N/A	N/A	2.58% val	1.24 min
Best Regularization (L2)	N/A	N/A	76.90% val	0.04 min
Best Optimizer (RMSProp)	N/A	N/A	88.72% val	0.03 min
Best Hyperparameters (C2)	89.67%	1.48%	88.58%	0.06 min
Fine-Tuned Model (C4)	46.40%	1.92%	N/A	3.64 min

14. Source Code

[GitHub Repository](#)

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Parameters are learned automatically during training, while hyperparameters are set manually beforehand.

2. Why is weight initialization important?

Initializing weights to zero causes gradients to be symmetric, and the neural network fails to learn anything meaningful right at the first epoch.

3. Why be concerned about zero initialization for neural networks?

Zero initialization causes all the gradients to be symmetric and all the neurons in each layer of the neural network to learn the same thing.

4. Compare Xavier and He initialization.

Xavier: suitable for using sigmoid/tanh, keeping the variance of the input and hidden layers approximately the same. He: suitable for using ReLU and considers the zero-thresholding behavior of ReLU.

5. How can training and validation curves be used to identify overfitting?

When we see the loss of the training set going down, while the loss of the validation set first goes down and then goes up.

6. How does Dropout reduce overfitting?

Dropout trains the neural network to become less sensitive to the deactivation of some neurons, thereby forcing the neural network to learn more robust and less redundant representations.

7. What is the purpose of Batch Normalization?

Batch normalization accelerates the process of training the neural network by reducing the internal covariate shift of the neural network.

8. Explain the numerical Batch Normalization example.

Normalize the input to have zero mean and unit variance: given the input $[2, 4, 6, 8]$, we first calculate the mean $\mu = 5$ and variance $\sigma^2 = 5$, then we linearly transform the input and get $[-1.34, -0.45, 0.45, 1.34]$.

9. What are the roles of γ and β ?

We use γ and β to rescale the input so that the representation capability of the neural network is not lost due to the transformation to zero mean and unit variance.

10. Compare SGD, Momentum, RMSProp and Adam.

Stochastic Gradient Descent is a basic optimization algorithm that takes steps in the direction of the negative gradient; Momentum helps to accelerate SGD in relevant directions through the accumulation of velocity; RMSProp considers the influence of gradient magnitude in different dimensions by taking a weighted average of the squared gradients and making appropriate adjustments to the learning rate; Adam combines the advantages of Momentum and RMSProp.

11. What happens when the learning rate is too large?

Optimization algorithms oscillate around the minimum and cannot converge.

12. What happens when the learning rate is too small?

The training process becomes excruciatingly slow, and the algorithm may tend to get stuck in local minima.

13. What is the effect of increasing batch size?

Increasing batch size leads to a better estimate of the gradient and faster iterations, but can generalize worse. A larger batch size is less likely to find good local minima because the noisy gradient updates are suppressed.

14. Explain stride and padding.

Stride is the step size by which the filter slides over the input; zero padding adds a border of zeros around the input, which maintains the spatial dimension of the output in the convolution.

15. Why is MobileNetV2 computationally efficient?

It uses inverted residual blocks and depthwise separable convolutions to drastically reduce the number of param-

eters and computational cost.

16. What is depthwise separable convolution?

A depthwise separable convolution splits the standard convolution operation into a depthwise convolution, which conducts convolutions on each input channel, and a 1×1 pointwise convolution.

17. What is transfer learning?

Transfer learning attempts to use the knowledge captured by a model that has been trained on one dataset to improve the performance of another model on a different but related task.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, we freeze all the trainable parameters of the pretrained network and train a new output layer on top of the pretrained network. In fine-tuning, we unfreeze some of the parameters of the pretrained network and further tune the pretrained network.

19. Why is a smaller learning rate generally used during fine-tuning?

A smaller learning rate is needed for fine-tuning so that the pretrained weights are not updated too drastically.

20. Why must the test set remain untouched during tuning?

We do not want any information from the test set to influence decisions during the hyperparameter tuning process.

21. Why should mean and standard deviation both be reported?

The mean reflects the general performance of the model, while the standard deviation reflects the variance of the model's performance.

22. Is the highest validation accuracy always sufficient to select a model?

No. A model that has a lower mean accuracy but a much lower standard deviation may be better than one with a higher mean accuracy but a large standard deviation.

16. Additional Exercise

Two new combinations of learning rate, dropout, batch size, and fine-tuning strategy are selected. These are evaluated using 5-fold cross-validation and their results compared with the selected configuration.

A justification is provided regarding whether the new configuration is preferable based on accuracy, standard deviation, and computational cost.

17. Expected Outcome

This experiment demonstrates that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>